

Toegepaste Informatica

2TX

2025-2026



UCLL
HOGESCHOOL



Full-stack software development

Front-end advanced: Hooks

J. Pieck, J. Rombouts, B. Van Impe

React/Next.js

JSX

Components

Props

Rendering

Hooks

Pages

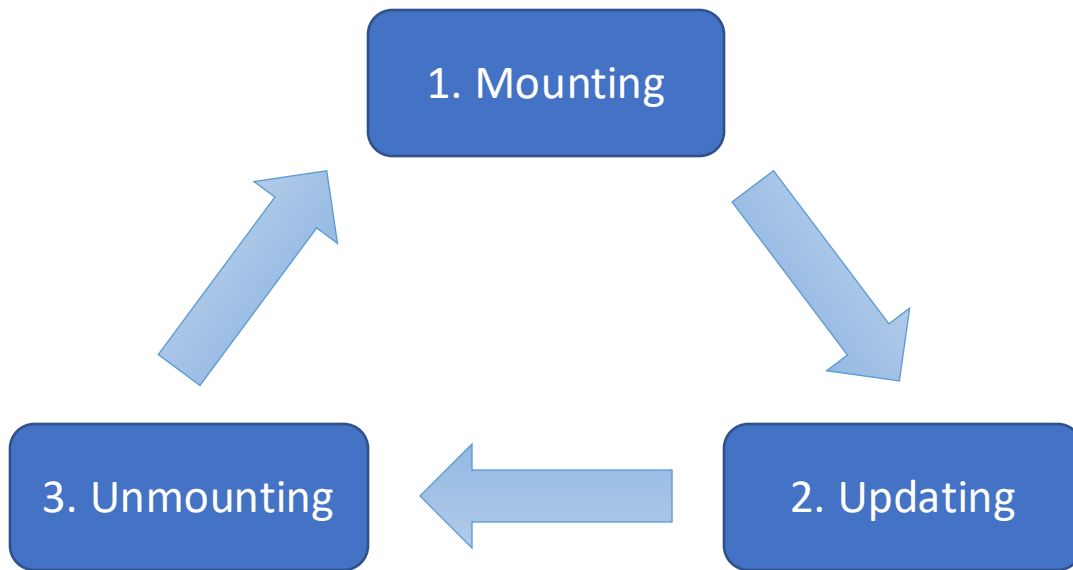
Routing

Hooks

- The lifecycle of a React component
- “Hook” into the React lifecycle
- useEffect and useState revisited
- Implement a polling mechanism with setInterval
- All hooks in these slides are only for client components.



Lifecycle phases of a React component



Understanding the React lifecycle is crucial when building React applications. To understand what **hooks** are, you need to understand the different lifecycle phases of a React component.

Rules of hooks

- Only call hooks at the top level of your component or from your own custom hook.
 - Not inside loops and conditions (React relies on a **consistent order** of hook calls on every component render to track state and effects correctly. If a hook's execution order changes between renders, React loses track of which state or effect belongs to which hook, leading to inconsistent behaviour and corrupted state.

useState hook revisited

- The most used hook.
- Since we now know that a component can be frequently re-rendered during the **update** phase, it might be necessary to “remember” information during different renders of a component.
- Why not just local variables?
 - React uses state to **optimize** re-rendering and doesn't consider local variables.
 - Local variables are function scoped, state is component scoped.
 - The values of local variables are lost after a re-render.

Anatomy of useState

```
const [username, setUsername] = useState<string>("");
```

useState returns an array of which the first value is the **state variable** and the second a **state setter function**. Here, we “destructure” those 2 values in two local variables “username” and “setUsername” which we can use in our component.

Specifies the type of the state variable.

The initial value which is set during the **mount** phase. Can be null.

useEffect

- A hook to perform side-effects that are outside the scope of the component rendering:
 - E.g. reading and updating browser storage
- Only call useEffect to interact with an external system, otherwise use state and event handlers.

Anatomy of useEffect

```
useEffect(() => { console.log("Component rendered!"); });
```



A **callback** function that is executed when the side-effect occurs

This call of useEffect has no **dependency array** as second parameter, so it is called after **every** render during **mount** and **update**.

```
useEffect(() => { console.log("Component mounted!"); }, []);
```



With an **empty** dependency array as second parameter: only executed once, after the first render, i.e. in the **mount** phase.

```
useEffect(() => { console.log("Effect ran because userName changed!"); }, [userName]);
```



With **one or more** elements in the dependency array: executed in **mount** phase and after every render in the **update** phase, but only when there is a change in one of the elements of the array (username in the example).

Implement polling with custom hook

```
// add polling to get all users every 5 seconds  
useInterval(getUsers, 5000);
```

- **useInterval** is a custom hook.
 - Leverages javascript **setInterval** behind the scenes.
- The first parameter of **useInterval** is a **callback** function that is executed after every interval specified by the second parameter.
- To install this hook: `npm install use-interval`

Why not directly setInterval?

- useInterval considers the component lifecycle.
 - The interval is cleared when the component unmounts.
 - Prevent unnecessary callback executions and possible memory leaks.
- <https://overreacted.io/making-setinterval-declarative-with-react-hooks/>

Polling with useSWR

```
useInterval(() => {  
  mutate("schedulesAndStudents", getSchedulesAndStudents());  
}, 1000);
```

- The result of useSWR is not directly stored in state, like with useEffect.
 - The **swr** library (useSWR) manages the **data** object.
- As a consequence, when using **useInterval**, you must tell swr which data to alter using the **mutate** function (swr library).
 - You have to use the same key that you have passed to useSWR.

Newer useRouter and usePathName hooks

- When using the App Router, you can use these 2 hooks from “next/navigation”
- This will give you an instance of the AppRouter.

```
const router = useRouter();
```

- You can use this router to navigate to other places in your app:

```
router.push("/");
```

Newer useRouter and useParamsName hooks

- useParamsName() returns a pathname that returns the value of your current url as a string.

```
const pathname = useParamsName()
```

- This doesn't sound very useful, but you can use it, e.g., to trigger a useEffect on the url change:

```
useEffect(() => { console.log("Pathname changed:", pathname); }, [pathname]);
```

References

- <https://www.javatpoint.com/react-component-life-cycle>
- <https://react.dev/reference/react>
- <https://react.dev/learn/state-a-components-memory>